

Explorador de Archivos en Python

Guía Completa por Niveles

Proyecto de aprendizaje para GitHub

30 de junio de 2026

Índice

1. Introducción	3
2. Nivel Principiante – Funciones esenciales	3
3. Nivel Intermedio – Funciones de gestión y análisis	6
3.1. Renombrar en lote avanzado	6
3.2. Mover archivos a otra carpeta	7
3.3. Copiar carpeta con barra de progreso	7
3.4. Deshacer / Rehacer simple	7
3.5. Papelera casera	7
3.6. Eliminación segura	8
3.7. Compresión y extracción	8
3.8. Comparar dos carpetas	8
3.9. Cálculo de hash de archivo	8
3.10. Buscar duplicados	9
3.11. Monitor de cambios en tiempo real (watchdog)	9
4. Nivel Avanzado – Funciones profesionales	9
4.1. Interfaz con pestañas y panel dual	9
4.2. Vista previa rápida (Quick Look)	10
4.3. Miniaturas de imágenes	10
4.4. Búsqueda por contenido (indexada)	10
4.5. Cifrado de archivos (AES)	10
4.6. Verificación de firma digital (Windows)	11
4.7. Integración con Git	11
4.8. Terminal embebida	11
4.9. Plugins	11
4.10. Graba macros de acciones	11
4.11. Mapa de disco (Treemap) en consola	11
4.12. Editor hexadecimal básico	12
4.13. Diferencias entre archivos de texto	12
4.14. Visor de metadatos de imágenes (EXIF)	12
4.15. Administrador de sesiones	12
4.16. Vista previa de fuentes	12

4.17. Reproductor multimedia	13
4.18. Conversión de imágenes por lote	13
5. Código completo: Explorador de archivos por consola	13

1. Introducción

Este documento recopila todas las funciones que puedes implementar en un explorador de archivos, desde las más básicas hasta las más avanzadas. Está organizado en tres niveles de dificultad y cada idea incluye una breve explicación y, en muchos casos, un fragmento de código Python de ejemplo. Al final se proporciona un programa completo que integra múltiples funcionalidades.

2. Nivel Principiante – Funciones esenciales

Comienza por aquí si estás dando tus primeros pasos con el módulo `os` y la manipulación de archivos.

1. Mostrar archivos de una carpeta

Lista el contenido de una ruta y muestra si es un archivo o una carpeta.

```
1 import os
2 def listar_archivos(ruta):
3     if os.path.exists(ruta):
4         for elemento in os.listdir(ruta):
5             tipo = "Archivo" if os.path.isfile(os.path.join(ruta,
6                                                         elemento)) else "Carpeta"
7             print(f"{tipo}: {elemento}")
8     else:
9         print("La ruta no existe")
```

2. Crear carpetas desde una lista

Crea todas las carpetas cuyos nombres vienen en una lista.

```
1 def crear_carpetas(lista):
2     for nombre in lista:
3         os.mkdir(nombre)
4         print(f"Carpeta '{nombre}' creada")
```

3. Contar archivos dentro de una carpeta

Cuenta únicamente los archivos (ignora las subcarpetas).

```
1 def contar_archivos(carpeta):
2     if os.path.exists(carpeta):
3         archivos = [f for f in os.listdir(carpeta) if os.path.isfile(
4             os.path.join(carpeta, f))]
5         return len(archivos)
6     return 0
```

4. Buscar archivos por extensión (.txt)

Filtra los elementos que terminan en `.txt`.

```
1 def buscar_txt(carpeta):
2     if os.path.exists(carpeta):
3         for f in os.listdir(carpeta):
4             if f.endswith(".txt"):
5                 print(f)
```

5. Renombrar archivos añadiendo prefijo “copia_”

Agrega el prefijo a todos los archivos de una carpeta.

```
1 def prefijo_copia(carpeta):
2     for f in os.listdir(carpeta):
3         ruta_original = os.path.join(carpeta, f)
4         if os.path.isfile(ruta_original):
5             nuevo_nombre = "copia_" + f
6             os.rename(ruta_original, os.path.join(carpeta,
7                 nuevo_nombre))
8         print(f"Renombrado: {f} -> {nuevo_nombre}")
```

6. Eliminar archivos vacíos (0 bytes)

Detecta y borra archivos que no tienen contenido.

```
1 def eliminar_vacios(carpeta):
2     for f in os.listdir(carpeta):
3         ruta = os.path.join(carpeta, f)
4         if os.path.isfile(ruta) and os.path.getsize(ruta) == 0:
5             os.remove(ruta)
6             print(f"Eliminado: {f}")
```

7. Organizador de archivos por extensión

Crea subcarpetas (Imágenes, Documentos, etc.) y mueve los archivos según su extensión.

```
1 def organizar(carpeta):
2     categorias = {
3         "Im\'agenes": [".jpg", ".png", ".gif"],
4         "Documentos": [".pdf", ".docx", ".txt"],
5         "M\'usica": [".mp3", ".wav"]
6     }
7     for cat, exts in categorias.items():
8         os.makedirs(os.path.join(carpeta, cat), exist_ok=True)
9     for f in os.listdir(carpeta):
10        ruta = os.path.join(carpeta, f)
11        if os.path.isfile(ruta):
12            for cat, exts in categorias.items():
13                if any(f.lower().endswith(ext) for ext in exts):
14                    shutil.move(ruta, os.path.join(carpeta, cat, f))
15                    break
```

8. Árbol de directorios (recursivo)

Muestra la estructura de carpetas con indentación.

```
1 def arbol(ruta, nivel=0):
2     print(" " * nivel + "|-- " + os.path.basename(ruta))
3     if os.path.isdir(ruta):
4         for elem in os.listdir(ruta):
5             arbol(os.path.join(ruta, elem), nivel+1)
```

9. Detector de tamaño de archivos

Clasifica cada archivo según su tamaño (bytes, KB, MB, etc.).

```
1 def clasificar_tamano(carpeta):
2     for f in os.listdir(carpeta):
```

```

3     ruta = os.path.join(carpeta, f)
4     if os.path.isfile(ruta):
5         tam = os.path.getsize(ruta)
6         if tam == 0: cat = "Vac\ 'io"
7         elif tam < 1024: cat = "Bytes"
8         elif tam < 1024**2: cat = "KB"
9         elif tam < 1024**3: cat = "MB"
10        else: cat = "GB"
11        print(f"{f}: {tam} bytes ({cat})")

```

10. Simulador de limpieza automática

Encuentra archivos vacíos y archivos pesados (>1 MB) y muestra un resumen.

```

1 def simulador_limpieza(carpeta):
2     vacios, pesados = 0, 0
3     for f in os.listdir(carpeta):
4         ruta = os.path.join(carpeta, f)
5         if os.path.isfile(ruta):
6             tam = os.path.getsize(ruta)
7             if tam == 0:
8                 vacios += 1
9                 print(f"Vac\ 'io: {f}")
10            elif tam > 1024**2:
11                pesados += 1
12                print(f"Pesado: {f} ({tam//1024} KB)")
13    print(f"Encontrados {vacios} vac\ 'ios y {pesados} pesados.")

```

11. Búsqueda con comodines

Encuentra archivos que coinciden con un patrón usando fnmatch.

```

1 import fnmatch
2 def buscar_con_patron(carpeta, patron):
3     if os.path.exists(carpeta):
4         for f in os.listdir(carpeta):
5             if fnmatch.fnmatch(f, patron):
6                 print(f)

```

12. Copiar archivos (copia real)

Copia un archivo conservando metadatos con shutil.copy2.

```

1 import shutil
2 def copiar_archivo(origen, destino):
3     shutil.copy2(origen, destino)
4     print(f"Copiado a {destino}")

```

13. Calcular tamaño total de una carpeta

Suma el tamaño de todos los archivos recursivamente.

```

1 def tamano_total(carpeta):
2     total = 0
3     for dirpath, dirnames, filenames in os.walk(carpeta):
4         for f in filenames:
5             fp = os.path.join(dirpath, f)
6             total += os.path.getsize(fp)
7     return total

```

14. Exportar lista de archivos a CSV

Guarda nombre, tamaño y fecha de modificación en un archivo CSV.

```
1 import csv, datetime
2 def exportar_csv(carpeta, archivo_salida):
3     with open(archivo_salida, 'w', newline='') as f:
4         writer = csv.writer(f)
5         writer.writerow(["Nombre", "Tama~no (bytes)", "Modificaci~n"])
6         for elem in os.listdir(carpeta):
7             ruta = os.path.join(carpeta, elem)
8             if os.path.isfile(ruta):
9                 stat = os.stat(ruta)
10                writer.writerow([elem, stat.st_size, datetime.
                                datetime.fromtimestamp(stat.st_mtime)])
```

15. Archivos más grandes / más antiguos

Muestra los N archivos con mayor tamaño o con la fecha de modificación más antigua.

```
1 def top_tamano(carpeta, n=5):
2     archivos = []
3     for f in os.listdir(carpeta):
4         ruta = os.path.join(carpeta, f)
5         if os.path.isfile(ruta):
6             archivos.append((f, os.path.getsize(ruta)))
7     archivos.sort(key=lambda x: x[1], reverse=True)
8     for nombre, tam in archivos[:n]:
9         print(f"{nombre}: {tam} bytes")
```

16. Registro de actividades (log)

Escribe cada operación importante en un archivo explorador.log.

```
1 import logging
2 logging.basicConfig(filename='explorador.log', level=logging.INFO,
3                     format='%(asctime)s - %(message)s')
4 def log_evento(mensaje):
5     logging.info(mensaje)
```

3. Nivel Intermedio – Funciones de gestión y análisis

Aquí se añaden capacidades como mover archivos en lote, deshacer/rehacer, compresión, papeleras, comparación de carpetas, etc.

3.1. Renombrar en lote avanzado

Permite reemplazar texto, añadir numeración secuencial, cambiar extensión a minúsculas, etc.

```
1 def renombrar_lote(carpeta, viejo_texto="", nuevo_texto="", numerar=
    False):
2     for i, f in enumerate(os.listdir(carpeta)):
3         ruta = os.path.join(carpeta, f)
4         if not os.path.isfile(ruta): continue
5         nombre, ext = os.path.splitext(f)
```

```

6         if viejo_texto:
7             nombre = nombre.replace(viejo_texto, nuevo_texto)
8         if numerar:
9             nombre = f"{i+1:03d}_{nombre}"
10        nuevo_nombre = nombre + ext.lower()
11        os.rename(ruta, os.path.join(carpeta, nuevo_nombre))

```

3.2. Mover archivos a otra carpeta

Función general que mueve archivos que coinciden con un patrón.

```

1 def mover_archivos(origen, destino, patron="*"):
2     import fnmatch
3     for f in os.listdir(origen):
4         if fnmatch.fnmatch(f, patron) and os.path.isfile(os.path.join(
5             origen, f)):
6             shutil.move(os.path.join(origen, f), os.path.join(destino, f))

```

3.3. Copiar carpeta con barra de progreso

Copia todos los archivos y carpetas mostrando un porcentaje.

```

1 def copiar_con_progreso(origen, destino):
2     total = sum(len(files) for _, _, files in os.walk(origen))
3     copiados = 0
4     for root, dirs, files in os.walk(origen):
5         for file in files:
6             src = os.path.join(root, file)
7             dst = os.path.join(destino, os.path.relpath(src, origen))
8             os.makedirs(os.path.dirname(dst), exist_ok=True)
9             shutil.copy2(src, dst)
10            copiados += 1
11            print(f"\rCopiando: {copiados}/{total}", end="")
12    print()

```

3.4. Deshacer / Rehacer simple

Mantiene una pila de acciones. Cada acción es una tupla (tipo, origen, destino).

```

1 historial = []
2 def deshacer():
3     if historial:
4         tipo, src, dst = historial.pop()
5         if tipo == 'mover':
6             shutil.move(dst, src)
7         elif tipo == 'copiar':
8             os.remove(dst)
9         print("Deshecho.")

```

3.5. Papelera casera

Mueve archivos a una carpeta `.papelera` en lugar de eliminarlos.

```

1 PAPELERA = os.path.expanduser("~/papelera")
2 def enviar_a_papelera(ruta):
3     os.makedirs(PAPELERA, exist_ok=True)
4     shutil.move(ruta, os.path.join(PAPELERA, os.path.basename(ruta)))

```

3.6. Eliminación segura

Sobrescribe el archivo antes de borrarlo.

```

1 import secrets
2 def borrado_seguro(ruta, pasadas=3):
3     with open(ruta, "ba+") as f:
4         length = f.tell()
5         for _ in range(pasadas):
6             f.seek(0)
7             f.write(secrets.token_bytes(length))
8     os.remove(ruta)

```

3.7. Compresión y extracción

Crear y descomprimir archivos ZIP.

```

1 def comprimir_zip(carpeta, salida):
2     import zipfile
3     with zipfile.ZipFile(salida, 'w') as zipf:
4         for root, dirs, files in os.walk(carpeta):
5             for file in files:
6                 zipf.write(os.path.join(root, file),
7                           os.path.relpath(os.path.join(root, file),
8                                              carpeta))
9 def extraer_zip(archivo, destino):
10     import zipfile
11     with zipfile.ZipFile(archivo, 'r') as zipf:
12         zipf.extractall(destino)

```

3.8. Comparar dos carpetas

Usa filecmp.dircmp para obtener diferencias.

```

1 import filecmp
2 def comparar_carpetas(dir1, dir2):
3     dc = filecmp.dircmp(dir1, dir2)
4     print("Solo en izquierda:", dc.left_only)
5     print("Solo en derecha:", dc.right_only)
6     print("Diferentes:", dc.diff_files)

```

3.9. Cálculo de hash de archivo

Para verificar integridad o buscar duplicados.

```

1 import hashlib
2 def hash_archivo(ruta, algo="sha256"):
3     h = hashlib.new(algo)

```



```

4     with open(ruta, "rb") as f:
5         while True:
6             bloque = f.read(65536)
7             if not bloque: break
8             h.update(bloque)
9     return h.hexdigest()

```

3.10. Buscar duplicados

Agrupar archivos por hash y mostrar los que tienen copias.

```

1 def encontrar_duplicados(carpeta):
2     hashes = {}
3     for root, dirs, files in os.walk(carpeta):
4         for f in files:
5             ruta = os.path.join(root, f)
6             try:
7                 h = hash_archivo(ruta)
8                 hashes.setdefault(h, []).append(ruta)
9             except: pass
10    for h, rutas in hashes.items():
11        if len(rutas) > 1:
12            print(f"Duplicados (hash {h[:10]}...):")
13            for r in rutas: print("    ", r)

```

3.11. Monitor de cambios en tiempo real (watchdog)

Requiere la librería watchdog. Observa una carpeta e imprime los eventos.

```

1 from watchdog.observers import Observer
2 from watchdog.events import FileSystemEventHandler
3 class Manejador(FileSystemEventHandler):
4     def on_modified(self, event): print(f"Modificado: {event.src_path}")
5     def on_created(self, event): print(f"Creado: {event.src_path}")
6     def on_deleted(self, event): print(f"Eliminado: {event.src_path}")
7
8 def vigilar(carpeta):
9     observer = Observer()
10    observer.schedule(Manejador(), carpeta, recursive=True)
11    observer.start()
12    try:
13        while True: time.sleep(1)
14    except KeyboardInterrupt:
15        observer.stop()
16    observer.join()

```

4. Nivel Avanzado – Funciones profesionales

Estas características convierten tu explorador en una herramienta potente.

4.1. Interfaz con pestañas y panel dual

Se puede implementar con Tkinter o PyQt. La idea es tener un Notebook para pestañas y dos paneles Frame con listas de archivos independientes.

4.2. Vista previa rápida (Quick Look)

Al presionar espacio, muestra el contenido del archivo seleccionado. Para texto:

```
1 def previsualizar_texto(ruta, lineas=20):
2     try:
3         with open(ruta, 'r', encoding='utf-8') as f:
4             for i in range(lineas):
5                 linea = f.readline()
6                 if not linea: break
7                 print(linea, end='')
8     except: print("No se pudo previsualizar.")
```

Para imágenes, usar PIL para mostrar una miniatura en una ventana.

4.3. Miniaturas de imágenes

Genera y cachea miniaturas para la vista de iconos.

```
1 from PIL import Image
2 def generar_miniatura(ruta, tama~no=(64,64), cache_dir=".thumb_cache"):
3     os.makedirs(cache_dir, exist_ok=True)
4     nombre_cache = os.path.join(cache_dir, hashlib.md5(ruta.encode()).
5         hexdigest() + ".png")
6     if not os.path.exists(nombre_cache):
7         img = Image.open(ruta)
8         img.thumbnail(tama~no)
9         img.save(nombre_cache)
10    return nombre_cache
```

4.4. Búsqueda por contenido (indexada)

Con Whoosh o simplemente recorriendo archivos de texto:

```
1 def buscar_en_archivos(carpeta, palabra):
2     for root, dirs, files in os.walk(carpeta):
3         for f in files:
4             if f.endswith('.txt'):
5                 ruta = os.path.join(root, f)
6                 with open(ruta, 'r', errors='ignore') as file:
7                     if palabra in file.read():
8                         print(f"Encontrado en {ruta}")
```

4.5. Cifrado de archivos (AES)

```
1 from cryptography.fernet import Fernet
2 def generar_clave(): return Fernet.generate_key()
3 def cifrar_archivo(ruta, clave):
4     f = Fernet(clave)
5     with open(ruta, 'rb') as file: datos = file.read()
6     cifrado = f.encrypt(datos)
7     with open(ruta + '.enc', 'wb') as file: file.write(cifrado)
```

4.6. Verificación de firma digital (Windows)

Usar PowerShell o la API de Windows. Ejemplo con subprocess:

```
1 import subprocess
2 def verificar_firma(ruta):
3     cmd = f'powershell "Get-AuthenticodeSignature {ruta}"'
4     resultado = subprocess.run(cmd, capture_output=True, text=True,
5                               shell=True)
6     print(resultado.stdout)
```

4.7. Integración con Git

Con GitPython:

```
1 from git import Repo
2 def estado_git(ruta_repo):
3     repo = Repo(ruta_repo)
4     print("Rama:", repo.active_branch)
5     print("Modificados:", [item.a_path for item in repo.index.diff(None)
6                           ])
7     print("Untracked:", repo.untracked_files)
```

4.8. Terminal embebida

Abrir una terminal en la carpeta actual:

```
1 def abrir_terminal_aqui(ruta=None):
2     if os.name == 'nt':
3         subprocess.Popen(['cmd.exe'], cwd=ruta)
4     else:
5         subprocess.Popen(['x-terminal-emulator'], cwd=ruta)
```

4.9. Plugins

Se pueden cargar scripts desde una carpeta plugins con importlib.

4.10. Graba macros de acciones

Guardar una lista de comandos en un archivo JSON y reproducirlos.

4.11. Mapa de disco (Treemap) en consola

Dibuja un gráfico simple usando caracteres:

```
1 def treemap_consola(carpeta):
2     total = tamaño_total(carpeta)
3     for root, dirs, files in os.walk(carpeta):
4         tam = sum(os.path.getsize(os.path.join(root, f)) for f in files)
5         barra = '#' * int(50 * tam / total)
6         print(f"{os.path.basename(root):<20} {barra} {tam//1024} KB")
```

4.12. Editor hexadecimal básico

```
1 def ver_hex(ruta, bytes_por_linea=16):
2     with open(ruta, 'rb') as f:
3         offset = 0
4         while True:
5             bloque = f.read(bytes_por_linea)
6             if not bloque: break
7             hex_str = ''.join(f'{b:02x}' for b in bloque)
8             ascii_str = ''.join(chr(b) if 32 <= b < 127 else '.' for b
9                               in bloque)
10            print(f"offset:08x {hex_str:<{bytes_por_linea*3}} {
                ascii_str}")
            offset += len(bloque)
```

4.13. Diferencias entre archivos de texto

```
1 import difflib
2 def diff_archivos(archivo1, archivo2):
3     with open(archivo1) as f1, open(archivo2) as f2:
4         diff = difflib.unified_diff(f1.readlines(), f2.readlines(),
5                                     fromfile=archivo1, tofile=archivo2)
6     print(''.join(diff))
```

4.14. Visor de metadatos de imágenes (EXIF)

```
1 from PIL import Image
2 from PIL.ExifTags import TAGS
3 def mostrar_exif(ruta):
4     img = Image.open(ruta)
5     exif = img._getexif()
6     if exif:
7         for tag, valor in exif.items():
8             nombre = TAGS.get(tag, tag)
9             print(f"{nombre}: {valor}")
```

4.15. Administrador de sesiones

Al cerrar, guardar las pestañas abiertas en un archivo JSON y al abrir restaurarlas.

4.16. Vista previa de fuentes

```
1 from PIL import ImageFont, ImageDraw, Image
2 def previsualizar_fuente(ruta_fuente, texto="Hola mundo", tama~no=40):
3     font = ImageFont.truetype(ruta_fuente, tama~no)
4     img = Image.new("RGB", (400, 100), "white")
5     ImageDraw.Draw(img).text((10,10), texto, font=font, fill="black")
6     img.show()
```

4.17. Reproductor multimedia

Usar pygame o vlc para reproducir audio/video en una vista previa.

4.18. Conversión de imágenes por lote

```
1 def convertir_imagenes(carpeta, formato_destino="png"):
2     for f in os.listdir(carpeta):
3         if f.lower().endswith((' .jpg', ' .jpeg', ' .bmp')):
4             img = Image.open(os.path.join(carpeta, f))
5             nombre_sin_ext = os.path.splitext(f)[0]
6             img.save(os.path.join(carpeta, f"{nombre_sin_ext}.{
                formato_destino}"))
```

5. Código completo: Explorador de archivos por consola

El siguiente programa reúne las funciones más relevantes en un menú interactivo. Puedes usarlo como base y expandirlo.

```
1 #!/usr/bin/env python3
2 """Explorador de archivos por consola - Proyecto de aprendizaje"""
3 import os, shutil, csv, hashlib, fnmatch, datetime, zipfile
4
5 def menu_principal():
6     print("\n=== EXPLORADOR DE ARCHIVOS ===")
7     print(" 1. Listar archivos")
8     print(" 2. Crear carpetas desde lista")
9     print(" 3. Contar archivos")
10    print(" 4. Buscar .txt")
11    print(" 5. Prefijo 'copia_'")
12    print(" 6. Eliminar vacios")
13    print(" 7. Organizar por extension")
14    print(" 8. Arbol de carpetas")
15    print(" 9. Clasificar por tama~no")
16    print("10. Simulador limpieza")
17    print("11. Buscar con patron")
18    print("12. Copiar archivo")
19    print("13. Tama~no total")
20    print("14. Exportar CSV")
21    print("15. Top mas grandes")
22    print("16. Mover archivos")
23    print("17. Comprimir ZIP")
24    print("18. Hash de archivo")
25    print("19. Duplicados")
26    print("20. Ver hex")
27    print("21. Diferencias texto")
28    print("22. Metadatos EXIF")
29    print("23. Salir")
30
31 def main():
32     while True:
33         menu_principal()
34         op = input("Elige: ")
```

```

35     if op == "1":
36         ruta = input("Ruta: ")
37         if os.path.exists(ruta):
38             for f in os.listdir(ruta):
39                 print(f)
40     elif op == "2":
41         nombres = input("Nombres separados por coma: ").split(',')
42         for n in nombres:
43             os.mkdir(n.strip())
44             print("Creado", n)
45     elif op == "3":
46         carpeta = input("Carpeta: ")
47         print("Archivos:", contar_archivos(carpetas))
48     elif op == "4":
49         carpeta = input("Carpeta: ")
50         buscar_txt(carpetas)
51     elif op == "5":
52         carpeta = input("Carpeta: ")
53         prefijo_copia(carpetas)
54     elif op == "6":
55         carpeta = input("Carpeta: ")
56         eliminar_vacios(carpetas)
57     elif op == "7":
58         carpeta = input("Carpeta: ")
59         organizar(carpetas)
60     elif op == "8":
61         carpeta = input("Carpeta: ")
62         arbol(carpetas)
63     elif op == "9":
64         carpeta = input("Carpeta: ")
65         clasificar_tamano(carpetas)
66     elif op == "10":
67         carpeta = input("Carpeta: ")
68         simulador_limpieza(carpetas)
69     elif op == "11":
70         carpeta = input("Carpeta: ")
71         patron = input("Patron (ej. *.pdf): ")
72         buscar_con_patron(carpetas, patron)
73     elif op == "12":
74         origen = input("Origen: ")
75         destino = input("Destino: ")
76         copiar_archivo(origen, destino)
77     elif op == "13":
78         carpeta = input("Carpeta: ")
79         print("Tama\~no total:", tamaño_total(carpetas), "bytes")
80     elif op == "14":
81         carpeta = input("Carpeta: ")
82         archivo = input("Archivo CSV: ")
83         exportar_csv(carpetas, archivo)
84     elif op == "15":
85         carpeta = input("Carpeta: ")
86         n = int(input("Cuantos: ") or 5)
87         top_tamano(carpetas, n)
88     elif op == "16":
89         origen = input("Origen: ")
90         destino = input("Destino: ")
91         patron = input("Patron (* para todos): ")
92         mover_archivos(origen, destino, patron)

```

```

93 elif op == "17":
94     carpeta = input("Carpeta a comprimir: ")
95     salida = input("Archivo .zip: ")
96     comprimir_zip(carpeta, salida)
97     print("Comprimido.")
98 elif op == "18":
99     ruta = input("Archivo: ")
100    print("SHA256:", hash_archivo(ruta))
101 elif op == "19":
102    carpeta = input("Carpeta: ")
103    encontrar_duplicados(carpeta)
104 elif op == "20":
105    ruta = input("Archivo: ")
106    ver_hex(ruta)
107 elif op == "21":
108    a1 = input("Archivo 1: ")
109    a2 = input("Archivo 2: ")
110    diff_archivos(a1, a2)
111 elif op == "22":
112    ruta = input("Imagen: ")
113    mostrar_exif(ruta)
114 elif op == "23":
115     break
116 else:
117     print("Opcion invalida")
118     input("Enter para continuar...")
119
120 if __name__ == "__main__":
121     main()

```

Listing 1: Explorador de archivos completo